

Flink 无状态cp效率测试

由 朱明创建, 最后修改于十一月 01, 2021 35 views since 15 Apr 2022

一、背景

以RTLP-SHOW项目为基础，测试checkpoint不同模式下的性能和效率表现，尽可能减少cp时长，并尝试摆脱对JM内存的依赖。

测试Flink版本基于 1.13.1

测试环境配置基于 <http://ultron.ads.qihoo.net/#/asset/flinkCluster> (flink 测试集群，8 cores & 8 slots & 24G memory per task Manager)

para	value
Checkpointing Mode	Exactly Once
State Backend	HashMapStateBackend
Interval	30s
Timeout	45s
Minimum Pause Between Checkpoints	0ms
Maximum Concurrent Checkpoints	1
Unaligned Checkpoints	Disabled
Persist Checkpoints Externally	Disabled
Tolerable Failed Checkpoints	0

二、使用外部存储

目前的Flink提供的API如下

```
1 //env.setStateBackend((StateBackend)new MemoryStateBackend());
2
3 env.setStateBackend(new HashMapStateBackend());
env.getCheckpointConfig().setCheckpointStorage(new FileSystemCheckpointStorage("viewfs
```

将旧的MemoryStateBackend 替换为 HashMapStateBackend + setCheckpointStorage的组合。

其中HashMapStateBackend的工作状态（working state）存储在TaskManagers中的J VM堆内存；而保存点状态（checkpoints state）由setCheckpointStorage指定。

FileSystemCheckpointStorage它将状态快照写入配置的文件系统中，它仅使用了一部分JobManager中的内存（非JVM）存储最小的云数据（Minimal metadata）。

其中cp参数如下

Overview		History	Summary	Configuration
Option		Value		
Checkpointing Mode		Exactly Once		
Checkpoint Storage		FileSystemCheckpointStorage		
State Backend		HashMapStateBackend		
Interval		30s		
Timeout		45s		
Minimum Pause Between Checkpoints		0ms		
Maximum Concurrent Checkpoints		1		
Unaligned Checkpoints		Disabled		
Persist Checkpoints Externally		Disabled		
Tolerable Failed Checkpoints		0		

具体cp时间如下

Overview		History	Summary	Configuration				
	ID	Status	Acknowledged	Trigger Time	Latest Acknowledgement	End to End Duration	Checkpointed Data Size	Processed (persisted) In-flight data
	187	COMPLETED	32/32	2021-11-01 16:45:12	2021-11-01 16:45:13	537ms	79.3 KB	0 B (0 B)
	186	COMPLETED	32/32	2021-11-01 16:44:42	2021-11-01 16:44:43	834ms	79.3 KB	0 B (0 B)
	185	COMPLETED	32/32	2021-11-01 16:44:12	2021-11-01 16:44:13	480ms	79.3 KB	0 B (0 B)
	184	COMPLETED	32/32	2021-11-01 16:43:42	2021-11-01 16:43:43	390ms	79.3 KB	0 B (0 B)
	183	COMPLETED	32/32	2021-11-01 16:43:12	2021-11-01 16:43:13	515ms	79.3 KB	0 B (0 B)
	182	COMPLETED	32/32	2021-11-01 16:42:42	2021-11-01 16:42:43	386ms	79.3 KB	0 B (0 B)
	181	COMPLETED	32/32	2021-11-01 16:42:12	2021-11-01 16:42:12	141ms	79.3 KB	0 B (0 B)
	180	COMPLETED	32/32	2021-11-01 16:41:42	2021-11-01 16:41:43	228ms	79.3 KB	0 B (0 B)
	179	COMPLETED	32/32	2021-11-01 16:41:12	2021-11-01 16:41:13	358ms	79.3 KB	0 B (0 B)
	178	COMPLETED	32/32	2021-11-01 16:40:42	2021-11-01 16:40:43	539ms	79.3 KB	0 B (0 B)

三、使用JM memory（添加外部存储）

同上面一样，目前的Flink提供的memory存储cp的 API如下

```
1 //env.setStateBackend((StateBackend)new MemoryStateBackend());
2 env.setStateBackend(new HashMapStateBackend());
3 env.getCheckpointConfig().setCheckpointStorage(new JobManagerCheckpointStorage());
```

JobManagerCheckpointStorage将checkpoint的状态数据存储在Jobmanager的堆中，默认一个任务的cp占用大小为5MB。

由于Flink集群已配置state.checkpoints.dir参数，所以本项目的JobManagerCheckpointStorage还是会调用外部存储，最终还是存储外部路径。

其中cp参数如下

Overview		History	Summary	Configuration
Option		Value		
Checkpointing Mode		Exactly Once		
Checkpoint Storage		JobManagerCheckpointStorage		
State Backend		HashMapStateBackend		
Interval		30s		
Timeout		45s		
Minimum Pause Between Checkpoints		0ms		
Maximum Concurrent Checkpoints		1		
Unaligned Checkpoints		Disabled		
Persist Checkpoints Externally		Disabled		
Tolerable Failed Checkpoints		0		

cp时间如下

OverviewHistorySummaryConfiguration

	ID	Status	Acknowledged	Trigger Time	Latest Acknowledgement	End to End Duration	Checkpointed Data Size	Processed (persisted) in-flight data
+	34	COMPLETED	32/32	2021-11-01 17:04:54	2021-11-01 17:04:54	397ms	79.3 KB	0 B (0 B)
+	33	COMPLETED	32/32	2021-11-01 17:04:24	2021-11-01 17:04:24	718ms	79.3 KB	0 B (0 B)
+	32	COMPLETED	32/32	2021-11-01 17:03:54	2021-11-01 17:03:54	289ms	79.3 KB	0 B (0 B)
+	31	COMPLETED	32/32	2021-11-01 17:03:24	2021-11-01 17:03:24	239ms	79.3 KB	0 B (0 B)
+	30	COMPLETED	32/32	2021-11-01 17:02:54	2021-11-01 17:02:54	82ms	79.3 KB	0 B (0 B)
+	29	COMPLETED	32/32	2021-11-01 17:02:24	2021-11-01 17:02:24	68ms	79.3 KB	0 B (0 B)
+	28	COMPLETED	32/32	2021-11-01 17:01:54	2021-11-01 17:01:54	219ms	79.3 KB	0 B (0 B)
+	27	COMPLETED	32/32	2021-11-01 17:01:24	2021-11-01 17:01:24	617ms	79.3 KB	0 B (0 B)
+	26	COMPLETED	32/32	2021-11-01 17:00:54	2021-11-01 17:00:54	371ms	79.3 KB	0 B (0 B)
+	25	COMPLETED	32/32	2021-11-01 17:00:24	2021-11-01 17:00:24	81ms	79.3 KB	0 B (0 B)

四、差别分析

这两种策略的具体差别如下（运行30分钟）

	FileSystemCheckpointStorage	JobManagerCheckpointStorage
checkpoint duration	MIN:90ms Avg:285ms Max:1s	MIN:30ms Avg:215ms Max:785ms
Async duration	70ms	1ms
Sync duration	11ms	145ms
JobManager(运行30分钟)		
JVM Metaspace	174/256(MB)	158/256(MB)
JVM	9.10GB	9.34GB

原因分析：Async duration差异 由于FS访问JM中的元数据（Minimal metadat）耗时较长，JMCP直接堆内存数据发送外部存储

Sync duration差异 （都使用HashMapStateBackend 进行快照，理论上没有差异，可能是因为JMCP需要将cp状态传输到JM的JVM堆中，网络传输增大了耗时）

结论：在使用外部存储并且是无状态CP时两种方式无明显差距，对于有状态CP以及无外部存储的JMCP后续进一步验证

无标签